

NYC TAXI URBAN MOBILITY DATA EXPLORER — TECHNICAL REPORT

PROBLEM FRAMING AND DATASET ANALYSIS	2
Describe the dataset and its context	2
Identify data challenges (e.g., missing fields, outliers, anomalies)	2
Explain assumptions made during data cleaning	3
Highlight one unexpected observation that influenced your design	3
SYSTEM ARCHITECTURE AND DESIGN DECISIONS	4
Architecture Overview	4
Stack and schema justification	4
Trade-offs	4
ALGORITHMIC LOGIC AND DATA STRUCTURES	5
Custom Implementation: QuickSort for Zone Ranking	5
Pseudo-code:	5
INSIGHTS AND INTERPRETATION	6
Insight 1: Rush Hour Congestion Patterns	6
Insight 2: Coverage Gap Analysis (Supply-Demand Imbalance)	6
Insight 3: Anomaly Detection (System & Economic Noise)	7
REFLECTION AND FUTURE WORK	7
Technical and team challenges	7
Future improvements	7
Screenshots (Visualization)	8

PROBLEM FRAMING AND DATASET ANALYSIS

Describe the dataset and its context

The dataset consists of CSV files provided by the NYC Taxi and Limousine Commission (TLC) containing historical records of yellow and green taxi trips. Each record represents a single trip and includes detailed fields such as pickup/drop-off, timestamps, trip distance, fare amounts, and location IDs corresponding to NYC Taxi Zones. This data is collected via authorized in-cab technology providers (TPEP/LPEP) and is primarily used by the city and researchers to analyze urban mobility and traffic patterns.

We have been given CSV files that are part of the NYC TLC Trip Record Data. One has trip records, and the other has records for the location/zone where the trips are occurring. Inside “yellow_tripdata_2019_01.csv”, we have attributes such as: VendorID, tpep_pickup_datetime / tpep_dropoff_datetime: Timestamp fields storing when a trip begins and ends (TPEP = Taxicab Passenger Enhancement Program systems), passenger_count, trip_distance (in miles), PULocationID / DOLocationID: TLC taxi zone IDs for pick-up and drop-off. Fare breakdown: fare_amount, extra, mta_tax, tip_amount (credit card tips only; cash tips are not included), tolls_amount (bridge/tunnel tolls), improvement_surcharge (\$0.30 for Taxicab Improvement Fund), total_amount, congestion_surcharge (empty for January 2019; law effective Feb 2, 2019)

Identify data challenges (e.g., missing fields, outliers, anomalies)

- There were two vendors that installed devices to track trip data inside the vehicles, and one had long time intervals. We expected to have all records from January 2019, but some records were from 2018 in different months.
- Negative fare_amount, total_amount, or trip_distance (invalid by business rules), or zero or negative passenger_count

- Outliers: trips with speed > 100 mph (GPS/timestamp noise), fare > \$100 for < 1 mile (suspicious economic patterns)
- tip_amount only captures credit card tips; cash tips are not recorded
- congestion_surcharge empty for January 2019 (not yet in effect)
- Large file size requires chunked processing (100k rows per batch)

Explain assumptions made during data cleaning

- Trips with fare_amount < 0 or total_amount < 0 are excluded (invalid)
- Trips with trip_distance <= 0 are excluded
- Trips with passenger_count <= 0 are excluded (a taxi trip implies at least 1)
- Rows missing PULocationID, DOLocationID, or tpep_pickup_datetime are dropped
- speed_mph > 100 is capped to NaN (treated as data error, not real)
- Zones with empty or invalid geometry are filtered out

Highlight one unexpected observation that influenced your design

Tip and congestion surcharge fields:

- tip_amount: Automatically populated for credit card tips only. Cash tips are not included in this record.
- total_amount: Final total charged to the passenger. Sum of all fare components, but does NOT include cash tips.
- congestion_surcharge: The \$2.50 NY State Congestion Surcharge for yellow taxis went into effect on February 2, 2019. Because our dataset is from January 2019, this field is empty or zero across all records.

Additional finding: ~5% of trips had average speed > 100 mph (impossible in NYC), likely due to GPS/timestamp noise. This led us to add speed capping as a feature engineering and custom anomaly detection for system noise (speed > 80 mph).

SYSTEM ARCHITECTURE AND DESIGN DECISIONS

Architecture Overview

See docs/architecture-diagram.png for a visual representation.

The system has a three-tier architecture:

- Frontend: HTML/CSS/JS with Leaflet.js (maps) and Chart.js (visualizations)
- Backend: Flask REST API with CORS, input validation, auth logic
- Storage: SQLite database with normalized schema (trips, taxi_zones, users, etc.)

Data flow: User → Dashboard → Security Layer → Flask API → SQLite. Data is populated by an ETL pipeline (CSV + Shapefile loaders → Cleaner → Feature Engineer → DAL).

Stack and schema justification

- SQLite: Zero-configuration, portable, adequate for demo/single-user. Trade-off: not ideal for high-concurrency production.
- Vanilla JS: Lightweight, no build step, fast iteration. Trade-off: less structure for very large UIs.
- Batch ETL: Historical data, not streaming; allows full data quality checks. Trade-off: not real-time.
- Normalized schema: Fact table (trips) with dimension tables (taxi_zones, time_dim, payment_types) for analytics and referential integrity. Indexes on pickup_date, pickup_hour, pickup_location_id, dropoff_location_id for fast filtering and joins.

Trade-offs

- Portability vs. scalability: SQLite chosen over PostgreSQL for ease of setup
- Simplicity vs. feature richness: Vanilla JS over React for quicker delivery
- Data quality vs. completeness: Aggressive cleaning may remove legitimate edge cases but improves dashboard reliability

ALGORITHMIC LOGIC AND DATA STRUCTURES

Custom Implementation: QuickSort for Zone Ranking

We manually implemented QuickSort (no built-in sort, sorted(), or sort_values) to rank taxi zones by coverage gap ratio (dropoffs / pickups) and identify underserved areas.

Problem: Identify zones where drop-off demand vastly exceeds pickup supply (ratio 2.0), then rank them by severity. This requires sorting without library help.

Approach:

1. For each location_id, compute pickup count and dropoff count
2. Where pickup > 0, compute ratio = dropoff / pickup
3. Collect zones with ratio > 2.0 into a list of dicts {location_id, gap_ratio}
4. Apply custom QuickSort on gap_ratio (descending)
5. Return top 10

Pseudo-code:

```

...

function quick_sort_zones(arr, key='gap_ratio'):
    if len(arr) <= 1:
        return arr

```

```

pivot = arr[len(arr) // 2]
left = [x for x in arr if x[key] > pivot[key]] // descending
middle = [x for x in arr if x[key] == pivot[key]]
right = [x for x in arr if x[key] < pivot[key]]
return quick_sort_zones(left, key) + middle + quick_sort_zones(right, key)
...

```

Time Complexity: $O(n \log n)$ average case

Space Complexity: $O(\log n)$ for the recursion stack (in-place partition not used; list comprehensions create new lists, so overall $O(n)$ for intermediate structures, but stack is $O(\log n)$)

Other custom logic (no built-ins for core ranking):

- Anomaly detection: speed_mph > 80 (system noise), fare > \$100 for < 1 mile (economic noise)
- Choke point detection: avg speed < 4.5 mph (slower than walking)
- Coverage gap identification: supply vs. demand imbalance per zone

INSIGHTS AND INTERPRETATION

Insight 1: Rush Hour Congestion Patterns

Derivation: `/api/trips/hourly` returns trip count and average speed by hour.

Visualization: Line chart (average speed by hour).

Interpretation: Identifies when trip speeds drop significantly (e.g., 7–9 am, 5–7 pm). Useful for optimal travel times and traffic policy.

Insight 2: Coverage Gap Analysis (Supply-Demand Imbalance)

Derivation: `/api/trips/gaps` returns zones ranked by dropoff/pickup ratio using custom QuickSort.

Visualization: Map and table of top underserved zones.

Interpretation: High ratio = more people arriving than leaving. Highlights areas

where demand outstrips supply and expansion may be needed.

Insight 3: Anomaly Detection (System & Economic Noise)

Derivation: Custom logic in algorithms.py and aggregators.py.

- System noise: speed_mph > 80
- Economic noise: trip_distance < 1 mile and fare_amount > \$100
- Choke points: avg speed < 4.5 mph

Visualization: Diagnostic report, anomaly counts in summary.

Interpretation: Flags data quality issues and possible meter/fraud patterns.

REFLECTION AND FUTURE WORK

Technical and team challenges

- Large dataset size required chunked ETL and careful memory management
- Shapefile reprojection (EPSG:2263 → EPSG:4326) needed for Leaflet compatibility
- Balancing custom algorithm requirements with the practical use of pandas for ingestion and cleaning
- Coordinating backend (Flask/SQLite) with frontend (Leaflet/Chart.js) and ensuring API contracts stayed consistent

Future improvements

- Machine learning for trip demand prediction by location/time
- Real-time streaming from NYC Open Data

- Mobile app for field use
- Spatial clustering (ML) to auto-group high-demand zones
- Deployment: Docker, cloud hosting, auto-scaling
- Add payment_type to distinguish cash vs. card for tipping analysis

Screenshots (Visualization)





